

UNIVERSITY OF TWENTE.



Babette Bleeker – 3265331

Yulisa Blok - 3283568

Contents

Introduction	3
Hardware	4
Data collection	5
AI	7
Installation	10
Conclusion.....	12
Bibliography	13

Introduction

For this project, we built an interactive gardening game that combines machine learning, sensor interaction and game development. We used a TSP sensor, which lets users control the game using touch gestures. With these gestures, the player can plant and grow their own flowers in a virtual garden.

We used an LSTM model to recognize these different gestures and convert them into actions within the game. This model was initially trained in Python to achieve the best possible performance.

What makes the project particularly interesting is that, instead of using a traditional keyboard or mouse, the game allows players to perform natural movements similar to real gardening. The only difference is that the gestures are performed on a touchpad rather than in real life, making the experience more immersive and authentic.

The biggest challenge we faced was training the LSTM model to accurately recognize the various gestures. We implemented five different hand gestures that the user had to perform, and some of them were very similar. This made it difficult for the prediction model to tell them apart, so we had to adjust and improve the model many times to get the best results.

In the game I've got a little garden in my heart, everything revolves around gardening flowers. The game guides the player through the entire process. It starts with digging a hole using a digging gesture, then planting a seed, filling the hole, watering the plant, and finally harvesting the flower. There are also special challenges, like watering the plant just right. If you give too much water, the flower will wilt, and this is shown directly on the screen. The player also receives feedback on whether the gesture was correctly recognized by the model, so they can try again if needed.

This game combines fun gameplay with natural movements, creating a playful and interactive experience that feels more realistic and engaging.

Hardware

The following hardware was used in the project:

- ESP-32
- Silicon pad with conductive strips
- Capacitive sensing module
- USB-A to Micro-USB controller cable

We chose to incorporate the TSP sensor to capture hand motion. We preferred the capacitive sensor over a camera because it is less invasive to the privacy of users and the surrounding environment. Additionally, capacitive sensors only respond to conductive material, whereas cameras can be affected by lighting and background conditions, making them less reliable.

In the setup (see Figure 1), the ESP-32 was used as the central communication hub between the capacitive sensor, the silicon pad with conductive strips, and the laptop. The capacitive sensing module and the silicon pad were pre-assembled components. The Micro-USB cable was used to power the capacitive sensor and to transmit data via the serial port.



Figure 1: in this image the setup with the used hardware is displayed.

Data collection

Process of data collection

To train our LSTM model, we needed data based on real hand gestures. We collected this data by asking six participants to perform specific gestures on the TSP sensor. The participants were found through the Hands-on AI project course and are all classmates. Each gesture matched one of the five gardening actions in the game: digging, planting, closing, watering and harvesting.

Before collecting any data, we explained the purpose of our project to the participants and asked for their consent. We made sure they understood what the data would be used for and that their information would be handled safely and only for this project. The participants gave their consent verbally and could withdraw their data. We stored all data anonymously, but maintained a separate, secure log linking participant identities to their anonymized codes for reference if needed.

We recorded six sessions for each gesture to make sure the model was trained on enough examples. Each participant performed every gesture three times. The participants performed the movement when a verbal cue was given. They were also asked to diversify the movement. The data we collected included frames of the touch patterns and movements on the sensor, which were stored as time series data for the LSTM model to learn from. For each participant, we made a separate JSON file to ensure that there was as little redundant data as possible.

This data collection was covered under the general ethics approval for this course, application number 240218, approved by the CIS ethics committee of the University of Twente.

Preprocessing of data

To prepare the data, we first merged all individual participant data files into a single JSON file. The data was augmented by adding random noise to it to increase the size of the dataset, effectively tripling the dataset. Augmenting the data enabled us to train a more robust model with higher accuracy. Then labels were encoded into a numerical integer to be able to train the LSTM. The data was then split randomly into training and testing data. In this split 30 percent of the data was used for testing. From the testing data, 30 percent of the data was randomly split into validation data to be able to validate the model. Then the data was split into batches of 32, and the data that did not fit into a batch was dropped.

Description of the data set

The data consists of recorded frames of hand motion. These frames were stored in a 1D array in a dictionary that was stored in a list. The characteristics of the data set are that all but one participant was female. The participants were between 18 and 23 years old. Of all participants all but one female participant was right-handed. In the dataset, the planting class has been oversampled by 20% to improve the classification of this class, since this class severely underperformed in comparison to the other classes. To ensure the LSTM could effectively recognize gestures performed with either hand, 50% of the gesture samples were collected from the left-handed participant. To be certain of the diversity of the training data of the left-handed participant, the participant was closely monitored and instructed to diversify the data.

AI

ML algorithms applied

The data we collected is temporal, meaning it contains information over time. In addition, the data is sequential. For each hand motion, the length of the temporal and sequential data varies. Based on these characteristics, we decided to use an LSTM, which is a type of Recurrent Neural Network. This is a good fit because it takes into account the temporal context when classifying hand motions and helps understand how hand position and movement evolve over time.

Another benefit of using LSTM is that it can handle input sequences of varying lengths, which is useful for achieving smooth and reliable real-world performance. The model ends with a dense layer using softmax activation to classify the gestures.

To further improve the model, we added a bidirectional LSTM layer. This layer processes the input in both forward and backward directions simultaneously, allowing the model to learn from both past and future time steps (Anishnama, 2023). We found that adding this layer increased the model's accuracy by 0.10.

To prevent overfitting, we used two types of regularization. First, we added dropout layers after the bidirectional LSTM layer and the dense layer. Dropout works by randomly disabling neurons during training, which improves the model's ability to generalize (Mudigonda, 2021).

The second type of regularization we used was L2 regularization. This was applied to both the LSTM and dense layers. L2 regularization penalizes large weights, which helps reduce overfitting (Tewari, 2024).

To optimize the model during training, we used the AdamW optimizer. This optimizer generalizes better and helps reduce overfitting. It also reduced the training time. For the loss function, we used Sparse Categorical Crossentropy, since we were dealing with multi-class classification using integer labels (Shiksha, 2023). The model was trained using supervised learning, which means we labeled the data before feeding it into the system.

Quality of the model

The model we chose has an accuracy of 0.8749 and a training loss 0.4232, see figure 2. To start off we decided on this model because the model had the highest accuracy we have found with our dataset. In addition, the training and validation loss of this model followed the same shape. Lastly, the curves of the loss and validation loss are close, almost on to of each other, and have somewhat stabilized. The model is a bit underfitting in the first 20 epochs, but this gets corrected in the next 30 epochs, see Figure 3.

We chose to train the model in 50 epochs, this was the best balance between learning capacity and generalization, see Figure 2. When training we observed a trade-off, if the model had a high accuracy on the validation data the model's validation and training loss would show some overfitting, however when we tripled the data this trade-off was smaller and vice versa.

The other reason we chose this model is that the true positive classifications in the confusion matrix of the model in all 5 categories were similar, see Figure 4. Which we found important is to prohibit the model from over-classifying a certain hand motion.

Having a model that classifies well is important for our installation because the classification of our model shapes the user's experience. In our installation, each correctly labeled movement shows a fitting animation. For example, a digging motion shows a hole being dug on screen. If there is frequent misclassification the user won't be able to see the plant grow, but a replacement video of the movement, for example when making the wrong digging motion a mole shows up and tells you to do better. Having the model often misclassified will show these types of videos more often and can break the narrative of the experience. This could result in frustration for the users. Ensuring correct classification is therefore crucial from a technical standpoint and for the integrity of the interactive story point improving a smooth user experience.

Training loss per epoch

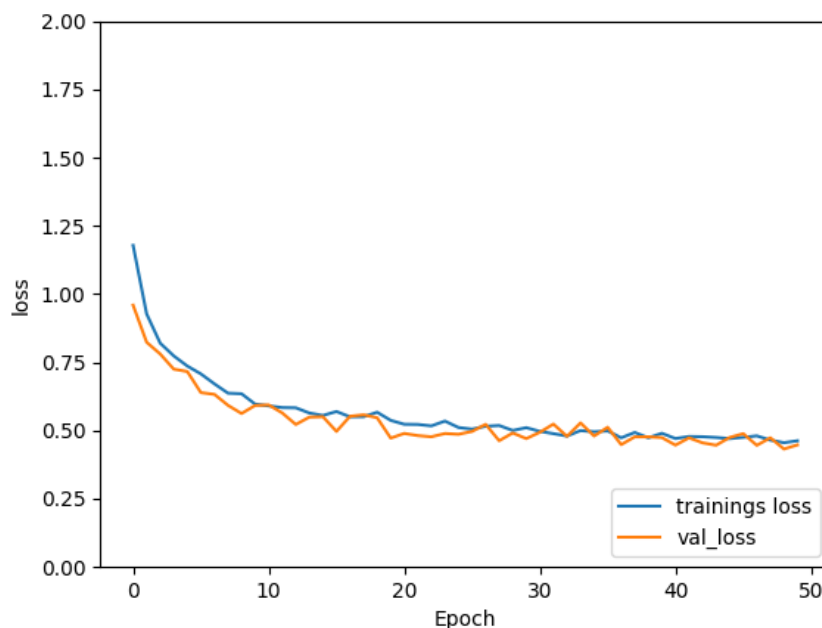


Figure 2: in this figure the training and validation loss are displayed. This figure shows that both of the loss is quite similar.

Accuracy per epoch

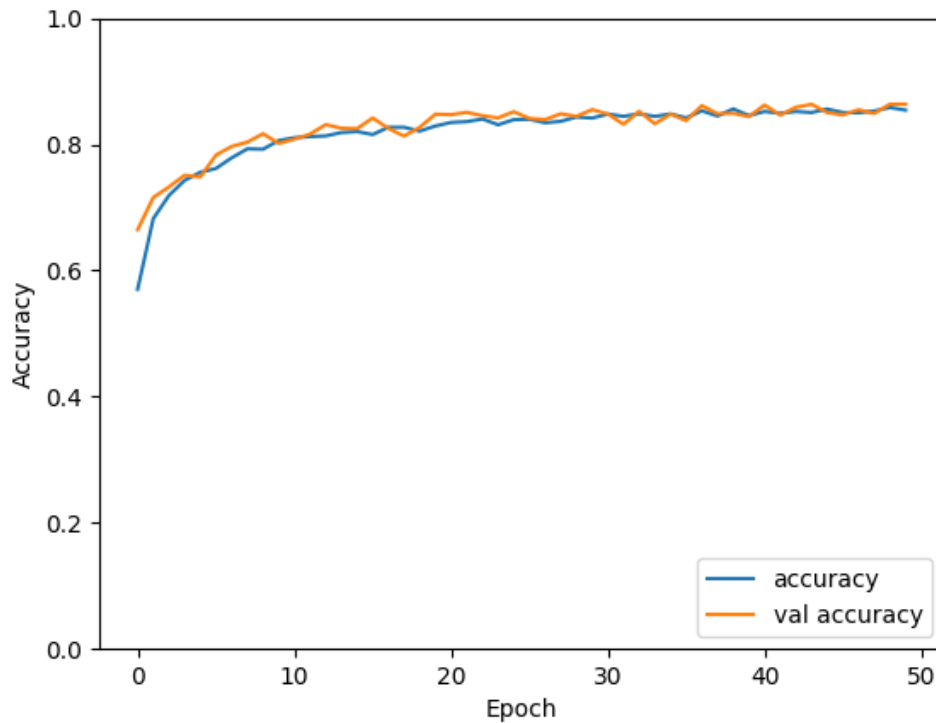


Figure 3: In this figure the accuracy per training epoch is displayed. The training accuracy and the validation accuracy are quite similar each epoch.

Confusion matrix

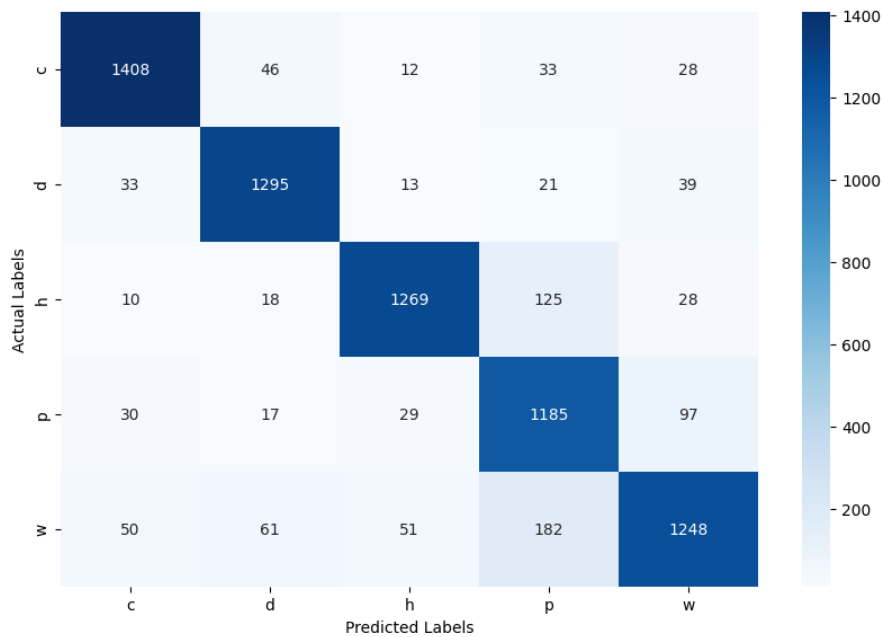


Figure 4 : in this figure the confusion matrix of the validation of the model is displayed. In the confusion matrix label C stand for closing, label d for digging, label h for harvesting, label p for planting and w for watering

Installation

Description of the Installation

Our game is called “I’ve Got a Garden in My Heart”, where you grow your own flowers using fun hand gestures. In the game, you use a TSP sensor to perform different hand motions and see the feedback directly on the screen. “I’ve Got a Garden in My Heart” is a playful and interactive game that lets players experiment with gestures in a creative and visual way. Every step in the process gives clear feedback, making it fun and rewarding to learn by doing. Whether your flower ends up perfect or a little muddy, you can always start again and enjoy growing something new.

- The system works as follows:
First, you start by digging a hole in the ground to prepare for planting. You make a digging motion on the TSP sensor, and if you do it correctly, a hole appears in the soil. If the game does not recognize your movement, a little mole pops up to let you know something went wrong. Once the hole is ready, you move on to planting the seed. If the gesture is recognized, the seed drops perfectly into the hole. If it is misclassified, the seed flies off and you need to try again.
- Next, it is time to water the seed. With each watering gesture, the plant grows bigger and the flower starts to bloom. But be careful, if you give too much water, the flower becomes droopy and loses its shape.
- Finally, you pick the flower by making a plucking gesture on the TSP. If the gesture is correct, the flower is placed nicely in a vase for you to enjoy. If it is misclassified, the flower falls into the mud and you have to try picking it again, this time ending up with a muddy flower. Once your flower is complete, you can start the process all over again and grow a brand-new one.

Evaluation

To evaluate the installation, we first tested it ourselves and then invited a few classmates to try it out. During initial testing, we found that the model consistently recognized the gestures for closing, digging, harvesting, and watering. However, the planting gesture was not always classified correctly.

To address this, we retrained the model using augmented data and added data of the planting motion. After retraining, we observed improved recognition of the planting gesture, especially when the gesture was performed in different areas of the silicone pad. We also noticed that the videos triggered by the gestures were displaying too slowly, which disrupted the user experience. As a result, we adjusted the delay times to make the video transitions more responsive.

Following these changes, we tested the updated installation with two additional participants. The model's accuracy had improved, and both users responded enthusiastically. They particularly appreciated the alternative feedback, such as the mole animation, when using the wrong hand motion. They found the gesture-based animations engaging. One point of feedback was that the system could respond a bit faster overall.

In the future, we plan to further enhance the experience by adding sound effects to accompany the animations, making the installation more immersive. Although the current model performs well, we believe additional data augmentation techniques—such as flipping and rotating gesture data—could further improve accuracy and robustness.

Conclusion

Looking back at the project, the main lesson we learned was how complicated and time-consuming the model fine-tuning process was. To be more specific, achieving a model that was neither underfitting nor overfitting, had good accuracy, and could classify all five hand motions was a bigger challenge than anticipated. This was underestimated during the planning phase and ended up taking more time than expected. Based on this experience, we would approach another deep learning (DL) model differently. In the future, we would start training with a smaller dataset, around 20 samples of each category to be classified, to build the general architecture. Having a smaller dataset means the model will run quicker, making changes and adjustments take less time. Then, we would add extra data and continue fine-tuning until we are satisfied with the model's metrics.

Another key takeaway was the importance of data preparation and diversity in the data. Initially, we trained the model with smaller data so that the model architecture could be designed and tuned to the dataset. However, we noticed that the dataset was not extensive enough and lacked variations. At first, we manually collected extra data, trying to vary the data as much as possible. However, this did not improve the model sufficiently. Therefore, we added data augmentation and tripled the dataset size.

We found the project interesting and enjoyed applying all the theory we had learned to create a final product. It was also great to see how all the different elements came together to form a working product. This really brought the theory to life and showed us how it can be applied in various ways. This experience helped us understand the concepts we studied even better, and we are proud of the project we created.

Bibliography

Anishnama. (2023, 18 mei). Understanding Bidirectional LSTM for Sequential Data Processing. *Medium*. [https://medium.com/@anishnama20/understanding-bidirectional-lstm-for-sequential-data-processing-b83d6283befc#:~:text=The%20Bi%2DLSTM%20layer%20processes,future%20\(following%20time%20steps\).](https://medium.com/@anishnama20/understanding-bidirectional-lstm-for-sequential-data-processing-b83d6283befc#:~:text=The%20Bi%2DLSTM%20layer%20processes,future%20(following%20time%20steps).)

Mudigonda, V. S. (2021, 26 december). Understanding dropout in deep neural networks - CodeX - Medium. *Medium*. <https://medium.com/codex/understanding-dropout-in-deep-neural-networks-95e7d1b11c58>

Online, S. (2023b, maart 31). *Cross entropy loss function in machine learning - Shiksha online*. Shiksha.com. <https://www.shiksha.com/online-courses/articles/cross-entropy-loss-function/#:~:text=for%20each%20sample.-,Sparse%20Categorical%20Cross%20Entropy%20Loss%20Function,than%20one%2Dhot%20encoded%20vectors.>

Tewari, U. (2024, 19 januari). Regularization — Understanding L1 and L2 regularization for Deep Learning. *Medium*. <https://medium.com/analytics-vidhya/regularization-understanding-l1-and-l2-regularization-for-deep-learning-a7b9e4a409bf>